SPIN36 — Resumen de implementación v1.1

Proyecto: SPIN36 · **Tecnología:** Android nativo con Kotlin + Jetpack Compose · **Versión:** 1.1

---

1. Autenticación con Firebase — Google Sign-In

Qué hace: Al abrir la app el usuario inicia sesión con su cuenta de Google en lugar de escribir su nombre manualmente. Firebase Authentication gestiona la identidad.

Cómo funciona:

- `BienvenidaScreen` muestra un botón "Iniciar sesión con Google"
- Al pulsar se lanza `GoogleSignInClient` que abre el selector de cuentas del sistema
- El token de Google se pasa a `FirebaseAuth.signInWithCredential()` que verifica la identidad
- El nombre del usuario viene directamente del perfil de Google (`account.displayName`)
- El UID único de Firebase identifica al jugador en toda la app

Cerrar sesión: Disponible en el menú de navegación de todas las pantallas. Llama a `FirebaseAuth.signOut()` y `GoogleSignIn.signOut()` y vuelve a la pantalla de bienvenida limpiando la pila de navegación.

Configuración necesaria:

- `google-services.json` con el proyecto Firebase
- Google Sign-In activado en Firebase Console → Authentication
- Cliente OAuth Web creado en Google Cloud Console
- SHA-1 del certificado de debug registrado en Firebase

---

2. Top Ten en Firebase Realtime Database con Retrofit + Moshi

Qué hace: Al terminar una sesión de juego, el saldo final del jugador se compara con el ranking global. Si entra en el top 10, su puntuación se guarda en la nube y recibe un premio de +500 monedas.

Arquitectura de la llamada REST:

```
JuegoViewModel
    └── TopTenRepository
            └── RetrofitClient (Retrofit + Moshi)
                    └── FirebaseApiService
                            ├── GET  top_ten.json         → leer ranking
                            └── PUT  top_ten/{uid}.json   → guardar puntuación
```

Firebase Realtime Database expone una API REST nativa. La URL base es:

```
https://spin36-dcae9-default-rtdb.europe-west1.firebasedatabase.app/
```

Moshi serializa y deserializa el modelo `PuntuacionDto` (nombre, puntuacion, fecha, uid) entre Kotlin y JSON automáticamente gracias a `@JsonClass(generateAdapter = true)` .

Retrofit construye las peticiones HTTP con la interfaz `FirebaseApiService` :

- `@GET("top_ten.json")` — obtiene todos los registros del top ten
- `@PUT("top_ten/{uid}.json")` — guarda o actualiza la puntuación del jugador

Lógica de entrada al top ten:

1. Se obtiene el top ten actual
2. Se comprueba si el jugador ya está, si hay menos de 10 entradas, o si supera el mínimo
3. Si entra → se guarda con `PUT` y se dan +500 monedas al jugador
4. Todo en `Dispatchers.IO` para no bloquear el hilo principal

3. Pantalla Top Ten

Qué hace: Muestra el ranking global de los 10 mejores jugadores ordenados por puntuación.

Características visuales:

- Posición #1 en dorado, #2 en plateado, #3 en bronce
- Cada card muestra: posición, nombre del jugador y puntuación
- Fondo verde casino con imagen de ruleta semitransparente
- Accesible desde el menú de navegación de todas las pantallas

---

4. Música de fondo

Qué hace: Reproduce música de fondo continua con control total por parte del usuario.

Implementación con `MediaPlayer`:

- Melodía oficial incluida en `res/raw/musica_oficial.mp3`
- Soporte para canción personalizada del dispositivo vía URI
- Control de volumen en tiempo real con slider
- **Audio Focus**: pausa automáticamente al recibir llamada u otro audio prioritario
- **Ducking**: baja el volumen a 0.2 ante notificaciones en lugar de parar del todo
- Pausa/reanudación automática al ir al background y volver (`onStart/onStop`)

---

5. Animación de la ruleta con sonido

Qué hace: Al pulsar GIRAR aparece un overlay que cubre la pantalla con una animación de números rodando que frena hasta pararse en el resultado.

Fase 1 — Números rodando:

- `LaunchedEffect` con bucle de intervalos crecientes: 70ms → 110ms → 180ms → 280ms → 400ms
- Simula el frenado de la ruleta
- Sonido `tick_ruleta.mp3` sincronizado en cada cambio de número con `SoundPool`

Fase 2 — Número final:

- **Si ganó:** estrella de 8 puntas dibujada con `Canvas` y trigonometría, con `BlurMaskFilter` para el difuminado dorado. Dos animaciones infinitas con `rememberInfiniteTransition`: rotación continua a 2400ms y pulso de escala de 0.6 a 1.15. Número en dorado. Sonido `win_sound.mp3` en bucle (`loop = -1`).
- **Si perdió:** sin estrella, número en negro, sin sonido de victoria.

El overlay no se puede cerrar hasta que la animación termina. El usuario toca para continuar.

---

6. Captura de imagen de la victoria

Qué hace: Al ganar, el usuario puede guardar una imagen personalizada de la victoria en su dispositivo.

Implementación:

- `GaleriaManager` genera un `Bitmap` con `Canvas` mostrando: nombre del jugador, número ganador, tipo de apuesta, cantidad apostada y monto ganado

- Todos los textos están traducidos al idioma activo del dispositivo
- `ActivityResultContracts.CreateDocument` abre el selector nativo del sistema para elegir ubicación
- Formato **WebP lossless** de alta calidad
- Al guardar suena `captura_sound.mp3`
- Tipografía `Mileast` aplicada al bitmap con `ResourcesCompat.getFont()`

7. Guardar victorias en el calendario

Qué hace: Al ganar, el usuario puede guardar un evento en el calendario nativo del dispositivo con los detalles de la victoria.

Implementación con `CalendarContract`:

- Content Provider nativo del SDK de Android
- Consulta `CalendarContract.Calendars` para obtener el ID del calendario principal
- Inserta en `CalendarContract.Events` con: título, descripción, fecha/hora exacta, zona horaria
- Operación en `Dispatchers.IO` para no bloquear el hilo principal
- Permisos: `READ_CALENDAR` y `WRITE_CALENDAR` solicitados en runtime

---

8. Notificaciones de victoria

Qué hace: Al ganar una apuesta se muestra una notificación con el tiempo que tardó el jugador en conseguir la victoria.

Implementación con `NotificationCompat`:

- Canal `canal_victoria` compatible con Android 8+ (API 26)
- Al pulsar la notificación vuelve a `MainActivity`
- Cierre automático (`autoCancel = true`)
- Permiso `POST_NOTIFICATIONS` solicitado en Android 13+

---

9. WebView con ayuda de la app

Qué hace: Pantalla de ayuda con la guía completa del juego en HTML local.

Implementación:

- `AyudaScreen` usa `AndroidView` con `WebView` de Android
- Carga HTML desde `assets/` según el idioma activo del dispositivo:
  - Español → `ayuda.html` , Inglés → `ayuda_en.html` , Catalán → `ayuda_ca.html` , Euskera → `ayuda_eu.html`
- JavaScript desactivado por seguridad
- HTML con diseño acorde a la app: fondo `#0F5C3A` , tipografía Cinzel, secciones con bordes dorados

---

10. Ubicación GPS en base de datos

Qué hace: Registra las coordenadas GPS del jugador en el momento de cada jugada.

Implementación con `FusedLocationProviderClient`:

- Obtiene la última ubicación conocida o solicita una nueva
- Latitud y longitud se guardan en `PartidaEntity` en Room (campos `Double?`)
- Operación en `Dispatchers.IO`
- Permisos: `ACCESS_FINE_LOCATION` y `ACCESS_COARSE_LOCATION`

11. Multiidioma — internacionalización y localización

Qué hace: La app soporta 4 idiomas, detecta el del dispositivo automáticamente y permite cambiarlo manualmente.

Idioma	Carpeta	Estado
Español	`values/strings.xml`	Base (defecto)
Inglés	`values-en/strings.xml`	Obligatorio
Catalán	`values-ca/strings.xml`	Adicional
Euskera	`values-eu/strings.xml`	Adicional

Todos los textos de la UI usan `stringResource(R.string.clave)`. Los strings del ViewModel usan `context.getString()`. El bitmap de captura recibe los textos ya traducidos como parámetros.

Cambio manual desde Ajustes con `ExposedDropdownMenuBox`. La selección llama a `AppCompatDelegate.setApplicationLocales()` que reinicia la Activity automáticamente y persiste la elección sin necesidad de `SharedPreferences`.

WebView también traducido: 4 archivos HTML, uno por idioma.

---

12. Sonido en botones

Qué hace: Todos los botones de la app reproducen un sonido al ser pulsados.

Implementación:

- `ButtonSoundPool`: singleton inicializado en `MainActivity.onCreate()`
- `rememberSoundClick(onClick)`: helper composable que envuelve cualquier `onClick` y reproduce `button_click.mp3` antes de ejecutarlo
- Aplica a todas las pantallas y a los ítems del menú de navegación

---

Resumen de tecnologías nativas del SDK usadas

Tecnología	Clase Android	Funcionalidad
Firebase Auth	`FirebaseAuth`	Autenticación con Google
Google Sign-In	`GoogleSignInClient`	Selector de cuenta
Retrofit + Moshi	`FirebaseApiService`	Top Ten vía REST
MediaPlayer	`MusicaManager`	Música de fondo
SoundPool	`SoundClickHelper`	Efectos de sonido
ContentResolver	`CalendarContract`	Guardar en calendario
FusedLocation	`UbicacionManager`	Coordenadas GPS
NotificationCompat	`NotificacionHelper`	Notificaciones

WebView	`AyudaScreen`	Pantalla de ayuda HTML
Canvas + BlurMaskFilter	`GaleriaManager` + `JuegoScreen`	Bitmap + animación estrella
AppCompatDelegate	`AjustesScreen`	Cambio de idioma en runtime